



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ**

**Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Лабораторная работа № 2
по дисциплине «Методы оптимизации»

**МЕТОДЫ СПУСКА
(0-го, 1-го и 2-го порядка и переменной метрики)**

Бригада 4

ПМ-34 РУБЦОВ АРТЁМ

ПМ-34 КУДЮКОВ МАКСИМ

Преподаватель

ЛЕМЕШКО БОРИС ЮРЬЕВИЧ

Новосибирск, 2026

Содержание	
1 Цель работы.....	3
2 Задание.....	3
3 Результаты	4
3.1 Квадратичная функция:	4
3.1.1 Выводы	7
3.2 Функция Розенброка:	8
3.2.1 Выводы	16
3.3 Целевая функция:	17
3.3.1 Выводы	22
4 Вывод	23
5 Программа	24
5.1 Main.cpp	24
5.2 MatrixOperations.h.....	24
5.3 MatrixOperations.cpp	24
5.4 Func.h	26
5.5 Func.cpp	26
5.6 MSGFR.h.....	27
5.7 MSGFR.cpp.....	28
5.8 Broyden.h	34
5.9 Broyden.cpp.....	34
5.10 VecOperations.h	40
5.11 VecOperations.cpp	40

1 Цель работы

Ознакомиться с методами поиска минимума функции n переменных в оптимизационных задачах без ограничений [1,5,7,8].

2 Задание

Метод	Порядок
Метод сопряженных градиентов в модификации Данилина-Пшеничного (Полака-Рибьера)	1
Метод Бroyдена	Переменной метрики
Метод квадратичной интерполяции (метод парабол)	

Условие задачи:

Найти максимум заданных функций:

На квадратичной функции: $f(x, y) = 100(y - x)^2 + (1 - x)^2$,

На функции Розенброка: $f(x, y) = 100(y - x^2)^2 + (1 - x)^2$,

На целевой функции:

$$f(x, y) = \frac{2}{1 + \left(\frac{x-1}{2}\right)^2 + \left(\frac{y-2}{1}\right)^2} + \frac{1}{1 + \left(\frac{x-3}{3}\right)^2 + \left(\frac{y-1}{3}\right)^2}$$

4 Вариант

A_1	A_2	a_1	a_2	b_1	b_2	c_1	c_2	d_1	d_2
2	1	1	3	2	3	2	1	1	3

3 Результаты

3.1 Квадратичная функция:

$$f(x, y) = 100(y - x)^2 + (1 - x)^2, \text{ eps} = 10^{-3}$$

Метод сопряженных градиентов в модификации Данилина-Пшеничного

Метод одномерного поиска: Золотое сечение

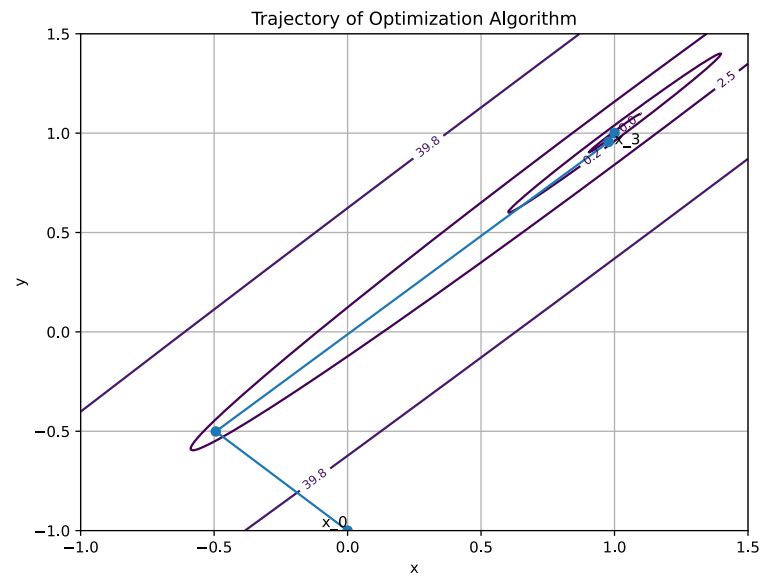
i	x_0 y_0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)
1	0 -1	-0.49379	-0.50122	2.236932	1.49014	1.497553	0.002494	0.49379	0.498778	98.76307	179.7142
2		0.977852	0.955704	0.049544	8.684238	17.36842	0.980247	1.471642	1.456926	2.187389	19.09114
3		1	1	2.45E-17	1.97E-08	-9.85E-09	0.00255	0.022148	0.044296	0.049544	71.56249
1	4 -1.6	1.192037	1.193	0.036971	-0.19156	-0.19252	0.002494	2.807963	2.793	3144.963	179.8799
2		1.000035	0.999969	4.35E-07	-0.01415	0.012265	1.002494	0.192002	0.193031	0.036971	94.09048
3		1	1	2.19E-20	1.36E-10	1.59E-10	0.002494	3.53E-05	3.06E-05	4.35E-07	4.442833

Метод сопряженных градиентов в модификации Данилина-Пшеничного

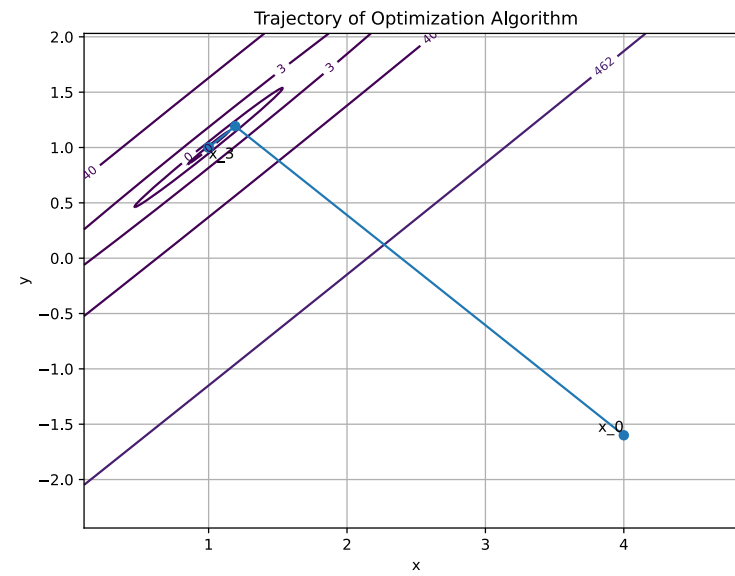
Метод одномерного поиска: Парабола (Отличий нет)

i	x_0 y_0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)
1	0 -1	-0.49379	-0.50122	2.236932	1.49014	1.497553	0.002494	0.49379	0.498778	98.76307	179.7142
2		0.977852	0.955704	0.049544	8.684238	17.36842	0.980247	1.471642	1.456926	2.187389	19.09114
3		1	1	2.45E-17	1.97E-08	-9.85E-09	0.00255	0.022148	0.044296	0.049544	71.56249
1	4 -1.6	1.192037	1.193	0.036971	-0.19156	-0.19252	0.002494	2.807963	2.793	3144.963	179.8799
2		1.000035	0.999969	4.35E-07	-0.01415	0.012265	1.002494	0.192002	0.193031	0.036971	94.09048
3		1	1	2.19E-20	1.36E-10	1.59E-10	0.002494	3.53E-05	3.06E-05	4.35E-07	4.442833

(0, -1)



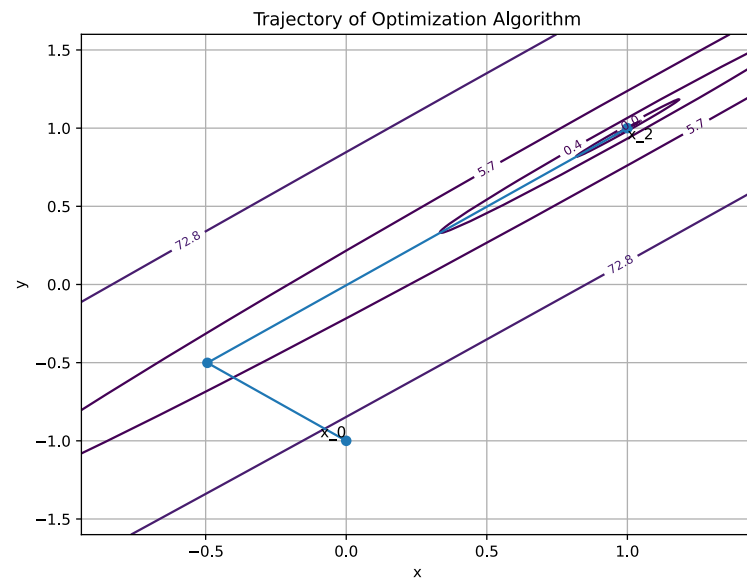
(4, -1.6)



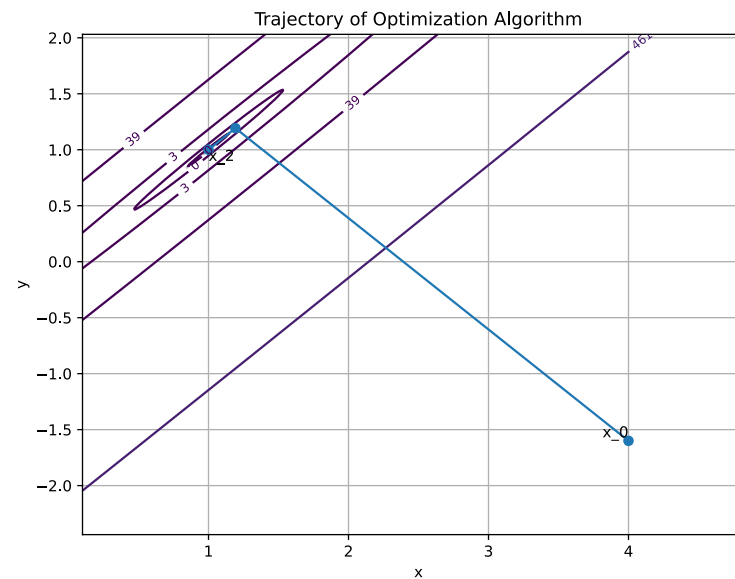
Метод Бройдена														
Метод одномерного поиска: Золотое сечение														
i	x_0 y_0	x_i	y_i	f_i	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	eta1	eta2	eta3	eta4	grad1	grad2
1	0	-0.4938	-0.5012	2.2369	0.0025	0.4938	0.4988	98.7631	0.4988	0.4987	0.4987	0.5037	-1.4988	-1.4888
2	-1	1.0000	1.0000	0.0000	1.0025	1.4938	1.5012	2.2369	0.5000	0.5000	0.5000	0.5050	0.0000	0.0000
1	4	1.1920	1.1930	0.0370	0.0025	2.8080	2.7930	3144.96	0.4988	0.4987	0.4987	0.5037	0.1818	0.2022
2	-1.6	1.0000	1.0000	0.0000	1.0025	0.1920	0.1930	0.0370	0.5000	0.5000	0.5000	0.5050	0.0000	0.0000

Метод Бройдена														
Метод одномерного поиска: Парабола (Отличий нет)														
i	x_0 y_0	x_i	y_i	f_i	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	eta1	eta2	eta3	eta4	grad1	grad2
1	0	-0.4938	-0.5012	2.2369	0.0025	0.4938	0.4988	98.7631	0.4988	0.4987	0.4987	0.5037	-1.4988	-1.4888
2	-1	1.0000	1.0000	0.0000	1.0025	1.4938	1.5012	2.2369	0.5000	0.5000	0.5000	0.5050	0.0000	0.0000
1	4	1.1920	1.1930	0.0370	0.0025	2.8080	2.7930	3144.96	0.4988	0.4987	0.4987	0.5037	0.1818	0.2022
2	-1.6	1.0000	1.0000	0.0000	1.0025	0.1920	0.1930	0.0370	0.5000	0.5000	0.5000	0.5050	0.0000	0.0000

(0, -1)



(4, -1.6)



3.1.1 Выводы

Оба метода сходятся к минимуму в точке (1, 1).

Метод Бройдена требует меньше итераций и сходится быстрее.

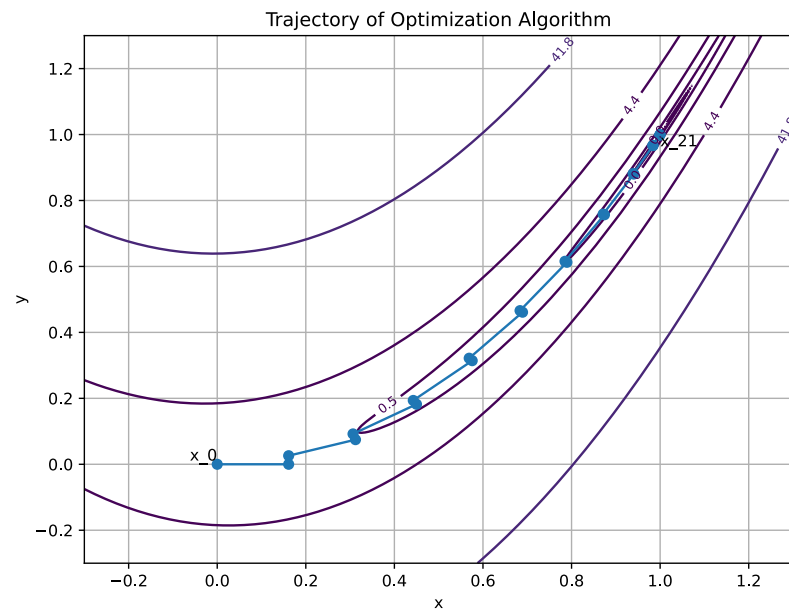
Выбор метода одномерного поиска (золотое сечение / парабола) практически не влияет на результат.

3.2 Функция Розенброка:

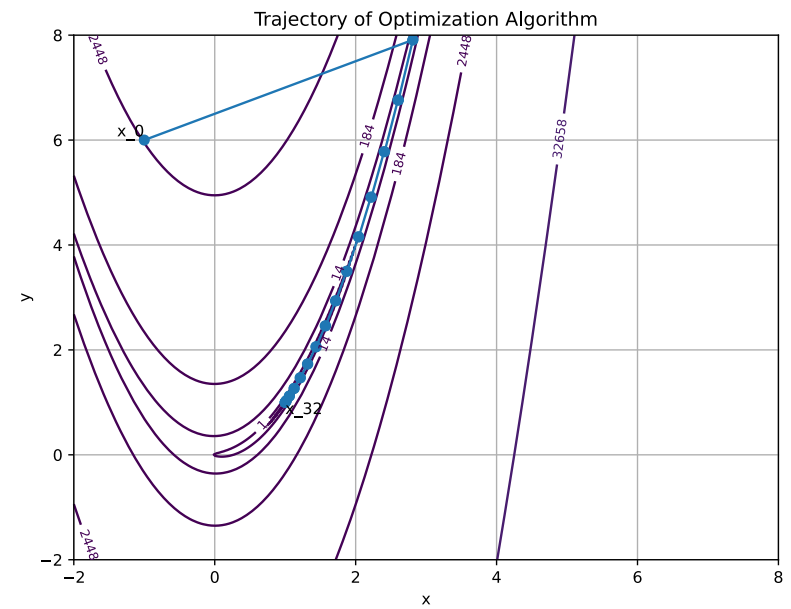
$$f(x, y) = 100(y - x^2)^2 + (1 - x)^2$$

Метод сопряженных градиентов в модификации Данилина-Пшеничного											
Метод одномерного поиска: Золотое сечение											
i	x_0 y_0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)
1	0 0	0.1613	0.0000	0.7711	13.5257	5.2011	0.0806	0.1613	0.0000	0.2289	21.0335
2		0.1613	0.0260	0.7035	1.6775	0.5410	0.0050	0.0000	0.0260	0.0676	8.7150
3		0.3122	0.0747	0.5250	12.1957	8.9641	0.0900	0.1509	0.0487	0.1785	22.8628
19		0.9994	0.9985	0.0000	0.2303	0.6839	2.3475	0.0160	0.0316	0.0003	26.4126
20		0.9993	0.9986	0.0000	0.0003	0.0006	0.0010	0.0001	0.0000	0.0000	18.4866
21		1.0000	1.0000	0.0000	-0.0001	0.0001	2.5016	0.0007	0.0014	0.0000	92.8668
1	-1 6	2.8118	7.9097	3.2838	0.3541	-0.7075	-0.0019	3.8118	1.9097	2500.7	133.8397
2		2.8119	7.9094	3.2837	-0.3111	-1.7494	0.0004	0.0002	0.0003	0.0001	170.5106
3		2.6078	6.7615	2.7400	-602.0683	-3129.4	0.6561	0.2041	1.1479	0.5437	169.7992
30		1.0008	1.0015	0.0000	-0.0003	-0.0006	0.0010	0.0001	0.0000	0.0000	161.5225
31		1.0000	1.0000	0.0000	-0.0003	0.0000	2.5058	0.0008	0.0015	0.0000	131.7373
32		1.0000	1.0000	0.0000	0.0000	0.0000	0.0010	0.0000	0.0000	0.0000	161.4594

(0,0)

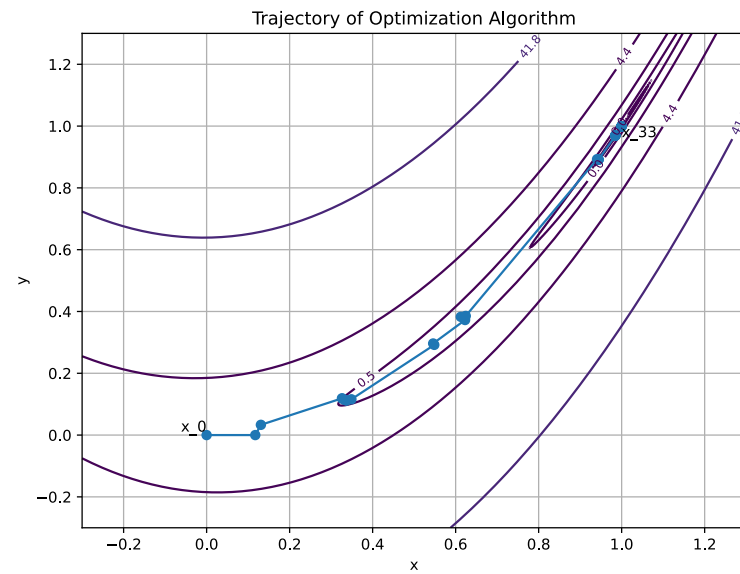


(-1, 6)

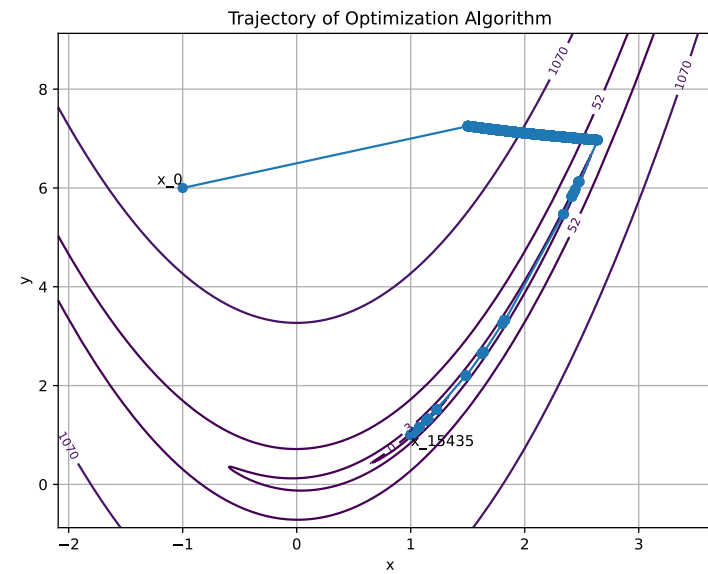


Метод сопряженных градиентов в модификации Данилина-Пшеничного											
Метод одномерного поиска: Парабола											
i	x_0 y_0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)
1	0 0	0.1172	0.0000	0.7982	5.5279	2.7491	0.0586	0.1172	0.0000	0.2018	26.4418
2		0.1308	0.0332	0.7814	4.7351	2.0785	0.0121	0.0135	0.0332	0.0168	9.4717
3		0.3261	0.1189	0.4699	7.2374	-0.6459	0.0413	0.1953	0.0857	0.3114	25.1318
31		0.9997	0.9993	0.0000	0.0003	0.0006	0.0023	0.0000	0.0000	0.0000	20.3291
32		0.9997	0.9993	0.0000	0.0002	0.0005	0.0018	0.0000	0.0000	0.0000	18.4608
33		1.0000	1.0000	0.0000	-0.0001	0.0000	1.3816	0.0003	0.0007	0.0000	95.9658
1	-1 6	1.5009	7.2530	2500.4	-1006.2	-3007.6	-0.0013	2.5009	1.2530	3.5517	173.1943
2		1.5011	7.2529	2499.9	6001.9	-2000.0	0.0000	0.0002	0.0001	0.5287	96.7362
3		1.5011	7.2529	2499.8	9002.8	-2999.8	0.0000	0.0000	0.0000	0.1355	96.7354
15433		1.0004	1.0008	0.0000	-0.0001	-0.0004	0.0012	0.0000	0.0000	0.0000	146.9167
15434		1.0004	1.0008	0.0000	-0.0036	-0.0071	0.0228	0.0000	0.0000	0.0000	161.6289
15435		1.0000	1.0000	0.0000	-0.0001	0.0000	0.1095	0.0004	0.0008	0.0000	118.8463

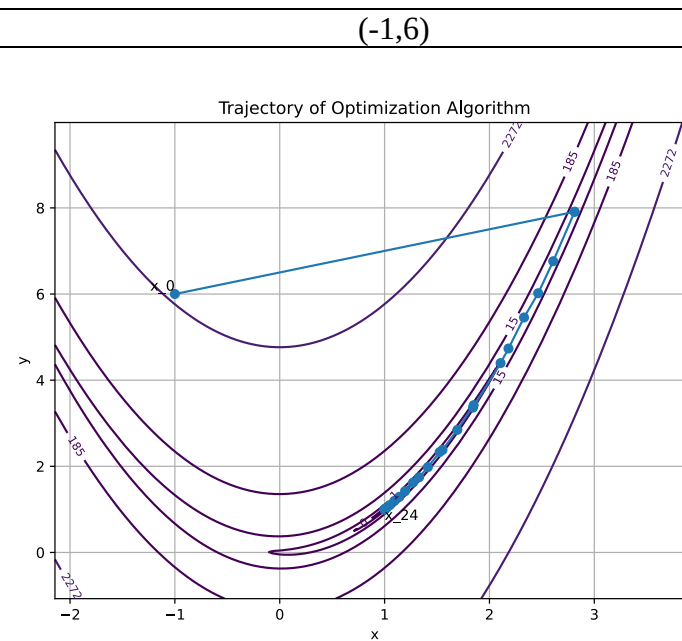
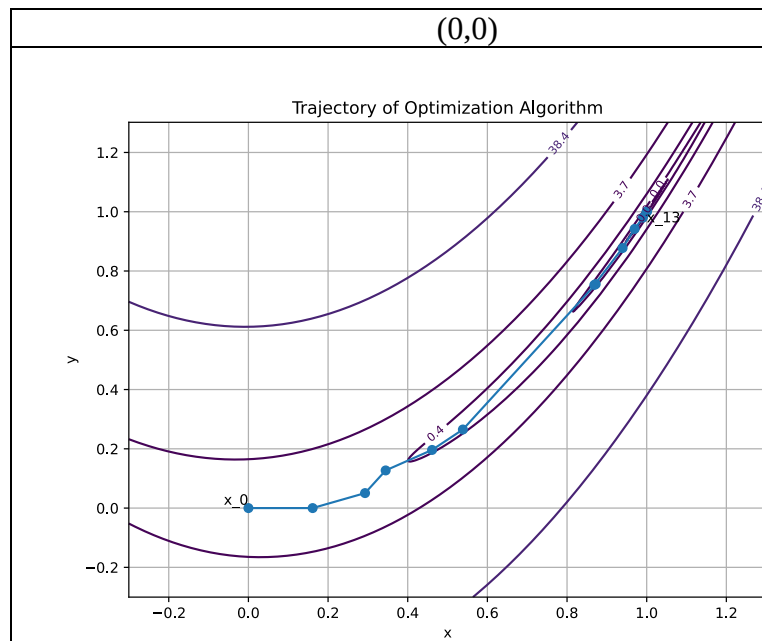
(0,0)



(-1, 6)

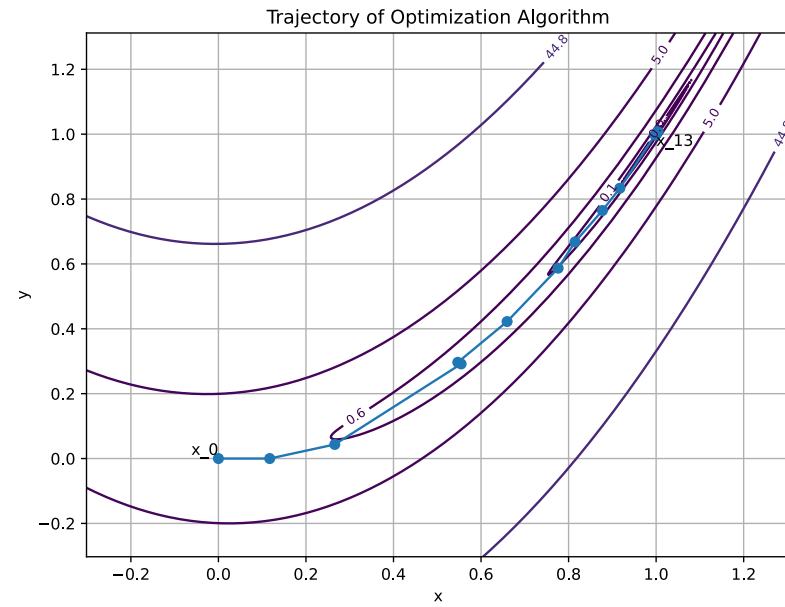


Метод Бройдена eps = 10 ⁻⁷														
Метод одномерного поиска: Золотое сечение														
i	x0 y0	x_i	y_i	f_i	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	eta1	eta2	eta3	eta4	grad1	grad2
1	0.0	0.1613	0.0000	0.7711	0.0806	0.1613	0.0000	0.2289	0.8900	0.3112	0.3112	0.1197	0.0000	-5.2011
2		0.2928	0.0506	0.6237	0.0813	0.1316	0.0506	0.1474	0.0741	0.0376	0.0376	0.0279	2.7039	-7.0316
3		0.3446	0.1271	0.4364	0.8085	0.0518	0.0765	0.1872	0.0372	0.0280	0.0280	0.0254	-2.4574	1.6637
11		1.0008	1.0014	0.0000	2.1737	0.0103	0.0211	0.0001	0.5038	0.9932	0.9932	1.9650	0.0821	-0.0402
12		0.9999	0.9999	0.0000	0.6126	0.0009	0.0015	0.0000	0.4409	0.8800	0.8800	1.7612	0.0011	-0.0006
13		1.0000	1.0000	0.0000	1.1481	0.0001	0.0001	0.0000	0.4992	0.9984	0.9984	2.0021	0.0000	0.0000
1	-1 6	2.8118	7.9097	3.2838	-0.0019	3.8118	1.9097	2500.7	0.1989	-0.4011	-0.4011	0.7992	-0.1474	0.6706
2		2.8120	7.9095	3.2838	0.0004	0.0001	0.0003	0.0001	-0.0001	-0.0011	-0.0011	-0.0049	0.9490	0.4756
3		2.6079	6.7620	2.7404	-341.9607	0.2040	1.1475	0.5434	0.1018	0.5531	0.5531	3.0079	44.2702	-7.8710
22		1.0008	1.0014	0.0000	1.3396	0.0117	0.0238	0.0002	0.4985	1.0028	1.0028	2.0228	0.0835	-0.0409
23		1.0002	1.0004	0.0000	1.0099	0.0006	0.0010	0.0000	-35.9641	-73.0634	-73.0634	148.4277	-0.0020	0.0012
24		1.0000	1.0000	0.0000	-0.0127	0.0002	0.0004	0.0000	0.5000	1.0002	1.0002	2.0058	0.0001	-0.0001

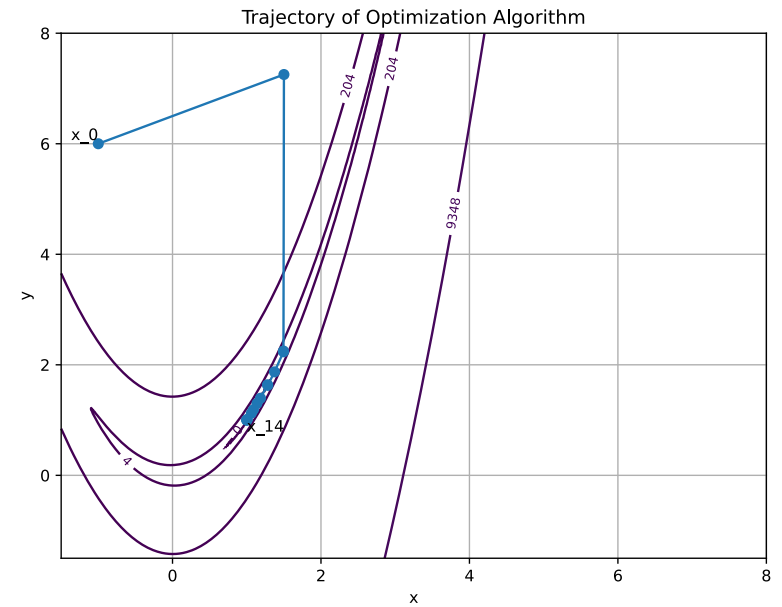


Метод Бройдена $\text{eps} = 10^{-7}$														
Метод одномерного поиска: Парабола														
i	x_0 y_0	x_i	y_i	f_i	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	eta1	eta2	eta3	eta4	grad1	grad2
1	0.0	0.1172	0.0000	0.7982	0.0586	0.1172	0.0000	0.2018	0.9295	0.2546	0.2546	0.0814	-1.1209	-2.7491
2		0.2659	0.0435	0.6131	0.0853	0.1486	0.0435	0.1850	0.1094	0.0483	0.0483	0.0295	1.4288	-5.4480
3		0.5545	0.2917	0.2233	2.7041	0.2886	0.2482	0.3898	0.1165	0.0661	0.0661	0.0743	2.6030	-3.1508
11		1.0045	1.0092	0.0000	0.3144	0.0068	0.0121	0.0003	0.2396	0.4668	0.4668	0.9134	-0.0619	0.0353
12		0.9999	0.9998	0.0000	2.8239	0.0047	0.0095	0.0000	0.5133	1.0283	1.0283	2.0650	-0.0204	0.0101
13		1.0000	1.0000	0.0000	0.9738	0.0001	0.0002	0.0000	0.4996	0.9990	0.9990	2.0026	0.0001	-0.0001
1	-1 6	1.5009	7.2530	2500.4	-0.0013	2.5009	1.2530	3.5517	-0.0005	-0.0002	-0.0002	1.0000	-3001.0	1000.0
2		1.4946	2.2496	0.2694	0.0050	0.0063	5.0033	2500.2	-0.0005	-0.0015	-0.0015	0.0005	-8.4138	3.1456
3		1.4951	2.2355	0.2451	0.9900	0.0005	0.0141	0.0242	-0.3635	-1.0869	-1.0869	-3.2449	0.8967	0.0313
12		1.0055	1.0109	0.0000	0.3802	0.0032	0.0077	0.0002	0.2454	0.4982	0.4982	1.0162	0.0665	-0.0276
13		1.0000	1.0000	0.0000	2.1450	0.0055	0.0109	0.0000	0.4935	0.9897	0.9897	1.9899	-0.0017	0.0009
14		1.0000	1.0000	0.0000	1.0176	0.0000	0.0000	0.0000	0.6049	1.2117	1.2117	2.4321	0.0000	0.0000

(0,0)



(-1,6)



3.2.1 Выводы

Оба метода находят минимум в точке (1, 1).

Метод сопряжённых градиентов сходится значительно медленнее и чувствителен к начальному приближению.

Метод Бройдена демонстрирует более быструю и устойчивую сходимость.

Метод параболы в одномерном поиске в ряде случаев требует больше итераций (вплоть до очень больших значений), что связано с узкой “долиной” функции Розенброка.

Золотое сечение показывает более стабильную, хотя и не всегда самую быструю сходимость.

3.3 Целевая функция:

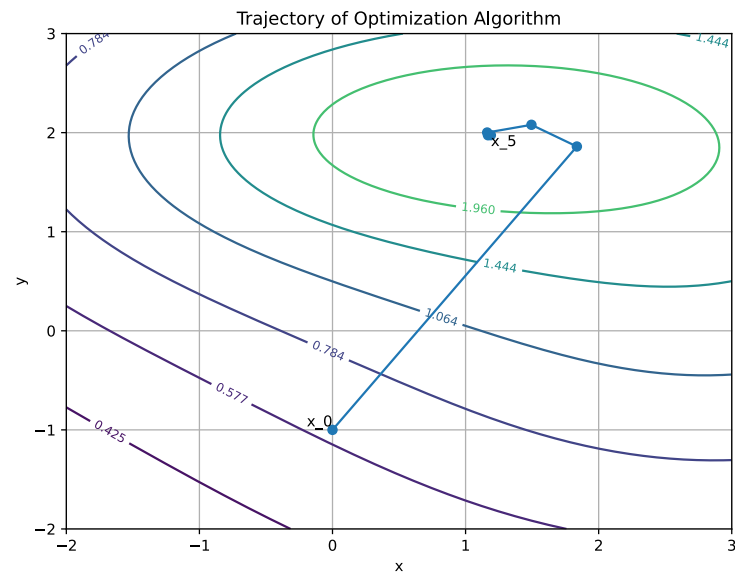
$$f(x, y) = \frac{2}{1 + \left(\frac{x-1}{2}\right)^2 + \left(\frac{y-2}{1}\right)^2} + \frac{1}{1 + \left(\frac{x-3}{3}\right)^2 + \left(\frac{y-1}{3}\right)^2}$$

Метод сопряженных градиентов в модификации Данилина-Пшеничного $\text{eps} = 10^{-7}$

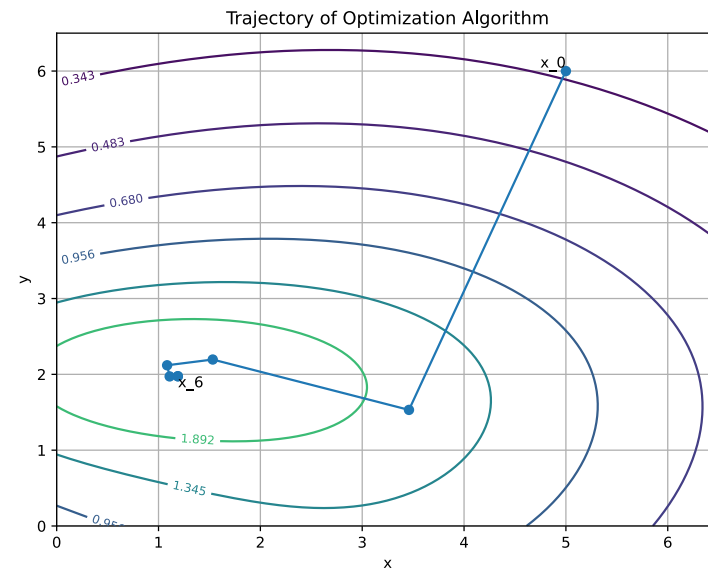
Метод одномерного поиска: Золотое сечение

i	x0 y0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)	grad1	grad2
1	0 -1	1.8362	1.8599	-2.4857	0.1730	1.1842	15.1640	1.8362	2.8599	1.8815	36.3227	0.4159	-0.2670
2		1.4948	2.0791	-2.5977	-0.6499	-0.1522	0.8207	0.3413	0.2192	0.1120	138.8982	0.2589	0.4033
3		1.1622	2.0012	-2.6596	-0.0084	-0.1130	0.5118	0.3326	0.0779	0.0619	154.1150	-0.0247	0.1053
4		1.1687	1.9736	-2.6611	0.0211	0.0009	0.2614	0.0064	0.0275	0.0015	57.0575	-0.0202	-0.0047
5		1.1899	1.9745	-2.6613	0.0000	-0.0002	1.0022	0.0212	0.0009	0.0002	145.9987	0.0000	0.0002
1	5 6	3.4591	1.5311	-1.6802	-1.0418	-1.6544	45.3195	1.5409	4.4689	1.3482	146.0753	0.4213	-0.1453
2		1.5324	2.1955	-2.5186	-1.6790	-0.2843	4.5735	1.9266	0.6643	0.8383	134.5263	0.2660	0.7715
3		1.0845	2.1196	-2.6148	-0.7386	-0.7087	0.2668	0.4480	0.0759	0.0962	160.9109	-0.0961	0.5673
4		1.1094	1.9726	-2.6582	0.0794	0.0025	0.2592	0.0249	0.1470	0.0434	58.8271	-0.0776	-0.0131
5		1.1897	1.9751	-2.6613	0.0002	-0.0027	1.0120	0.0804	0.0026	0.0031	145.1439	-0.0001	0.0027
6		1.1898	1.9744	-2.6613	0.0001	0.0000	0.2513	0.0000	0.0007	0.0000	59.5409	-0.0001	0.0000

(0, -1)

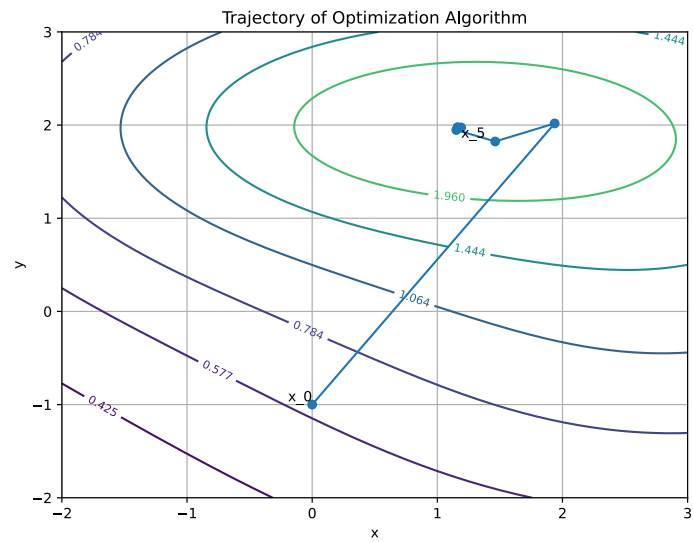


(5,6)

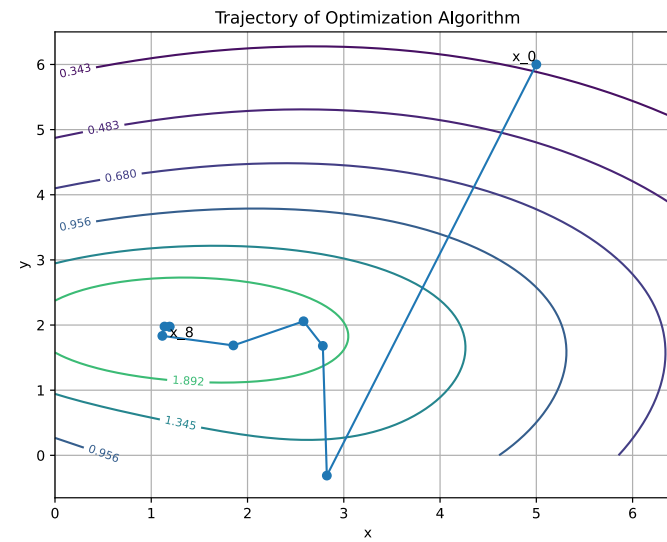


Метод сопряженных градиентов в модификации Данилина-Пшеничного $\text{eps} = 10^{-7}$													
Метод одномерного поиска: Парабола													
i	x0 y0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)	grad1	grad2
1	0 -1	1.9374	2.0176	-2.4455	0.1615	0.7994	16.0000	1.9374	3.0176	1.8413	32.4150	0.4764	0.1942
2		1.4624	1.8240	-2.5916	-0.7198	0.2856	0.9971	0.4750	0.1936	0.1461	107.0813	0.2024	-0.4965
3		1.1516	1.9473	-2.6591	0.0040	0.1249	0.4318	0.3108	0.1233	0.0675	28.7883	-0.0386	-0.1112
4		1.1620	1.9772	-2.6609	0.0285	-0.0029	0.2695	0.0104	0.0300	0.0019	65.3569	-0.0264	0.0091
5		1.1899	1.9744	-2.6613	0.0000	0.0002	0.9771	0.0279	0.0028	0.0004	32.5081	0.0000	-0.0002
1	5 6	2.8239	-0.3109	-1.1161	-0.4689	-0.9525	64.0000	2.1761	6.3109	0.7840	109.9241	0.0080	-0.3839
2		2.7823	1.6799	-2.0009	-0.4659	0.8797	5.1855	0.0417	1.9908	0.8848	86.7859	0.4521	-0.2207
3		2.5819	2.0582	-2.1020	-1.1644	0.9394	0.4300	0.2004	0.3783	0.1011	102.5419	0.5251	0.2674
6		1.1376	1.9771	-2.6600	0.0506	-0.0023	0.2653	0.0226	0.1433	0.0407	62.6421	-0.0499	0.0068
7		1.1900	1.9747	-2.6613	-0.0001	-0.0011	1.0351	0.0523	0.0023	0.0013	153.8151	0.0001	0.0011
8		1.1900	1.9744	-2.6613	-0.0001	0.0000	0.2520	0.0000	0.0003	0.0000	118.5455	0.0001	0.0000

(0, -1)

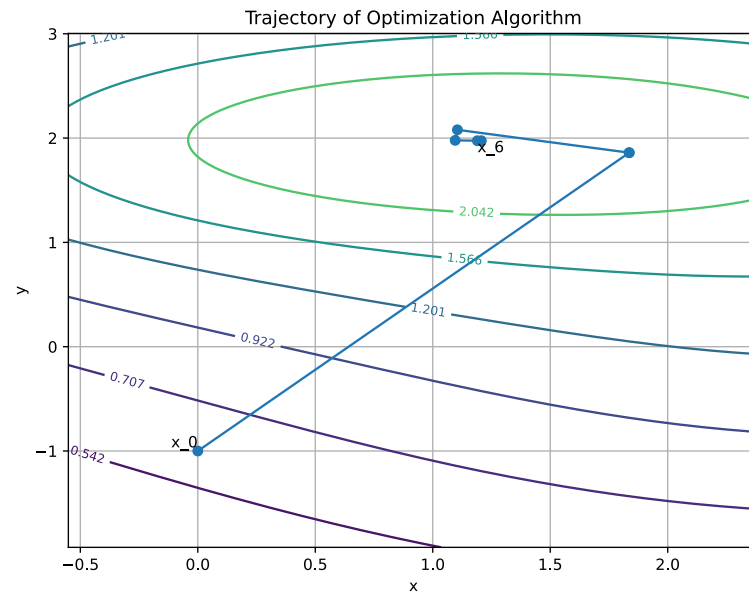


(5,6)

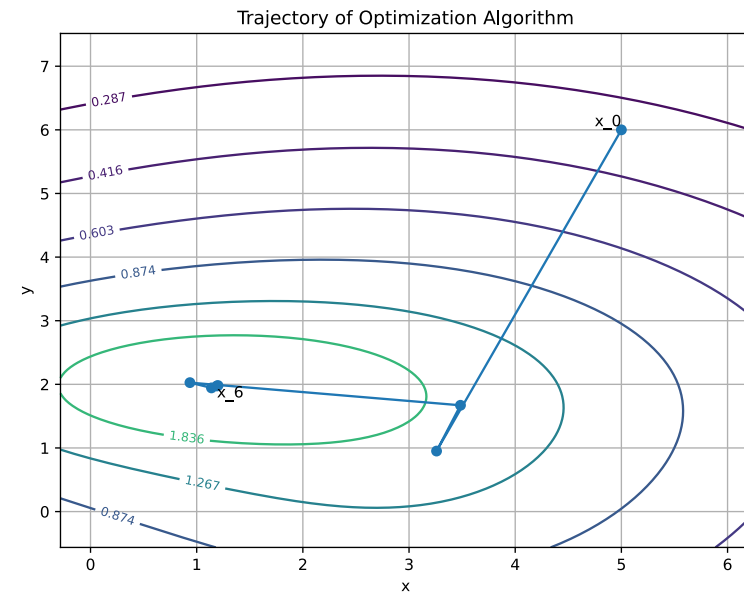


Метод Бройдена														
Метод одномерного поиска: Золотое сечение/ Парабола (разницы нет)														
i	x0 y0	x_i	y_i	f_i	s_1	s_2	λ	$ x_i - x_{i-1} $	$ y_i - y_{i-1} $	$ f_i - f_{i-1} $	Угол между (x_i, y_i) и (s_1, s_2)	grad1	grad2	i
1	0 -1	1.8362	1.8599	2.4857	15.1640	1.8362	2.8599	1.8815	4.6128	8.1710	8.1710	19.4799	0.4159	-
2		1.8362	1.8599	2.4857	0.0000	0.0000	0.0000	0.0000	1.0409	-0.0903	-0.0903	0.3728	0.4159	-
3		1.1049	2.0793	2.6363	1.6000	0.7313	0.2194	0.1506	2.0945	0.4426	0.4426	0.6424	0.0772	0.4142
4		1.0955	1.9781	2.6570	0.4361	0.0094	0.1012	0.0207	1.5296	-0.0281	-0.0281	0.2503	0.0908	0.0085
5		1.2068	1.9744	2.6612	0.8000	0.1113	0.0037	0.0041	1.0413	-0.0178	-0.0178	0.2500	0.0160	0.0011
6		1.1899	1.9744	2.6613	1.0198	0.0169	0.0000	0.0001	1.0622	-0.0198	-0.0198	0.2502	0.0000	0.0001
1	5 6	3.2591	0.9513	1.5848	51.2000	1.7409	5.0487	1.2527	3.2007	5.1281	5.1281	12.9493	0.2550	-
2		3.4844	1.6708	1.6836	0.2000	0.2253	0.7195	0.0988	0.9128	0.1579	0.1579	2.1527	0.4464	-
3		0.9362	2.0269	2.6253	6.4000	2.5482	0.3561	0.9417	3.7916	0.2839	0.2839	2.1583	0.2449	0.1973
4		1.1390	1.9441	2.6581	0.2324	0.2027	0.0828	0.0328	3.0594	1.2108	1.2108	0.9850	0.0511	-
5		1.2005	1.9810	2.6612	0.2000	0.0615	0.0370	0.0031	1.0297	-0.0122	-0.0122	0.2482	0.0105	0.0270
6		1.1899	1.9744	2.6613	1.0055	0.0105	0.0066	0.0001	1.0400	-0.0115	-0.0115	0.2483	0.0001	0.0001

(0, -1)



(5,6)



3.3.1 Выводы

Для заданной целевой функции оба метода находят точку оптимума.

Метод сопряжённых градиентов требует большего числа итераций.

Метод Бройдена достигает решения быстрее.

Методы одномерного поиска (парабола и золотое сечение) дают близкие результаты, однако метод параболы может обеспечивать более быстрый шаг при удачной аппроксимации, а золотое сечение — более устойчивое поведение.

4 Вывод

Оба реализованных метода позволяют эффективно решать задачи безусловной оптимизации. Метод сопряжённых градиентов является более универсальным, но требует большего числа итераций и вычислений функции. Метод Бройдена, благодаря использованию аппроксимации матрицы Гессе, обеспечивает более быструю и стабильную сходимость. Метод золотого сечения в одномерном поиске показывает более надёжное и устойчивое поведение на сложных функциях. Метод параболы может быть быстрее, но его эффективность зависит от формы функции и может ухудшаться на плохо обусловленных задачах.

5 Программа

5.1 Main.cpp

```
#include "MSGFR.h"
#include "Broyden.h"
#include <iostream>
#include <iomanip>
int main( )
{
    MSGFR msgfr;
    Broyden broyden;
    msgfr.Init( 2 );
    double *res;
    res = msgfr.Solver( );
    for ( int i = 0; i < 2; i++ )
        std::cout<<std::setprecision(16) << res[i] << std::endl;
}
```

5.2 MatrixOperations.h

```
#pragma once
class MatrixOperations
{
public:
    int n = 2;
    double **AddMatrix( double **mat1, double **mat2 );
    double **MultVecVecT( double *vec1, double *vec2 );
    double **MultMatScal( double **mat, double scal );
    double *MultMatVec( double **mat, double *vec );
};
```

5.3 MatrixOperations.cpp

```
#include "MatrixOperations.h"

double **MatrixOperations::AddMatrix( double **mat1, double **mat2 )
{
    double **res = new double *[n];
    for ( int i = 0; i < n; i++ )
    {
```



```

        res[i] = new double[n];
        for ( int j = 0; j < n; j++ )
            res[i][j] = mat1[i][j] + mat2[i][j];
    }
    return res;
}

double **MatrixOperations::MultVecVecT( double *vec1, double *vec2 )
{
    double **res = new double *[n];
    for ( int i = 0; i < n; i++ )
    {
        res[i] = new double[n];
        for ( int j = 0; j < n; j++ )
            res[i][j] = vec1[i] * vec2[j];
    }
    return res;
}

double **MatrixOperations::MultMatScal( double **mat, double scal )
{
    double **res = new double *[n];
    for ( int i = 0; i < n; i++ )
    {
        res[i] = new double[n];
        for ( int j = 0; j < n; j++ )
            res[i][j] = mat[i][j] * scal;
    }
    return res;
}

double *MatrixOperations::MultMatVec( double **mat, double *vec )
{
    double *res = new double[n] {0};
    for ( int i = 0; i < n; i++ )
        for ( int j = 0; j < n; j++ )
            res[i] += mat[i][j] * vec[j];
    return res;
}

```

5.4 Func.h

```
#pragma once
#include "VecOperations.h"
#include "MatrixOperations.h"
#include <iostream>
class Func
{
    VecOperations opsVec;
    MatrixOperations opsMat;
public:
    int n=2;
    double func(double *x );
    double *gradFunc(double *x );
    double funcInDirection(double *xk, double lambda, double *sk );
    double funcInDirectionEta( double *xk, double lambda, double **etak);
};
```

5.5 Func.cpp

```
#include "Func.h"

double Func::func(double* x)
{
    return 100 * ( x[1] - x[0] ) * ( x[1] - x[0] ) + ( 1 - x[0] ) * ( 1 - x[0] );
    //return 100 * ( x[1] - x[0]* x[0] ) * ( x[1] - x[0]* x[0] ) + ( 1 - x[0] ) * ( 1 - x[0] );
    //return -2 / ( 1 + ( x[0] - 1 ) * ( x[0] - 1 ) / 4.0 + ( x[1] - 2 ) * ( x[1] - 2 ) ) - 1 / ( 1 + ( x[0] - 3 ) * ( x[0] - 3 )
    // / 9.0 + ( x[1] - 1 ) * ( x[1] - 1 ) / 9.0 );
}

double* Func::gradFunc(double* x)
{
    double *res = new double[2] { -200 * ( x[1] - x[0] ) - 2 * ( 1 - x[0] ), 200 * ( x[1] - x[0] ) };
    //double *res = new double[2] { -400 * x[0]*( x[1] - x[0]*x[0] ) - 2 * ( 1 - x[0] ), 200 * ( x[1] - x[0]*x[0] ) };
    //double x1 = x[0], x2 = x[1];

    /////// Знаменатели (те же, что в исходной функции)
    //double g1 = 1.0 + ( x1 - 1.0 ) * ( x1 - 1.0 ) / 4.0 + ( x2 - 2.0 ) * ( x2 - 2.0 );
    //double g2 = 1.0 + ( ( x1 - 3.0 ) * ( x1 - 3.0 ) + ( x2 - 1.0 ) * ( x2 - 1.0 ) ) / 9.0;

    /////// Производные исходной (положительной) функции
    //double dfdx_plus = -( x1 - 1.0 ) / ( g1 * g1 ) - 2.0 * ( x1 - 3.0 ) / ( 9.0 * g2 * g2 );
}
```

```

//double dfdy_plus = -4.0 * ( x2 - 2.0 ) / ( g1 * g1 ) - 2.0 * ( x2 - 1.0 ) / ( 9.0 * g2 * g2 );

///// Для отрицательной функции градиент меняет знак
//double *res = new double[2];
//res[0] = -dfdx_plus; // минус от производной исходной
//res[1] = -dfdy_plus;

return res;
}

double Func::funcInDirection(double* xk, double lambda, double* sk)
{
    return func( opsVec.AddVec( xk, opsVec.MultVecScal( sk, lambda ) ) );
}

double Func::funcInDirectionEta( double *xk, double lambda, double **etak )
{
    return func( opsVec.AddVec( xk, opsVec.MultVecScal( opsMat.MultMatVec( etak, gradFunc( xk ) ), -lambda ) ) );
}

```

5.6 MSGFR.h

```

#pragma once
#include "Func.h"
#include "VecOperations.h"
#include <iostream>
class MSGFR
{
    int n1 = 2;
    int n;
    double *sk;
    double *sk_1;
    double lambdak;
    double *x0;
    double *xk;
    double *xk_1;
    double eps = 1e-7;
    double eps1 = 1e-14;
    Func f;
    VecOperations ops;
public:
    void Init( int n );
    void FindInterval( double& a, double& b );
    double *Solver( );
    double CalcW( );

```

```

double Min( double eps, double an, double bn );
double GoldenRatio( double eps, double an, double bn );
double Parabola( double eps, double an, double bn );
};

```

5.7 MSGFR.cpp

```

#include "MSGFR.h"
#include "iostream"
#include <iomanip>
#include <fstream>
#include <cmath>

void MSGFR::Init( int n1 )
{
    n = n1;
    x0 = new double[2] { 5, 6 };
    sk = ops.MultVecScal( f.gradFunc( x0 ), -1 );
    sk_1 = new double[n];
    xk = new double[n];
    xk_1 = new double[n];
    xk_old = new double[n];
    for ( int i = 0; i < n; i++ )
    {
        xk[i] = x0[i];
        xk_1[i] = x0[i];
        xk_old[i] = x0[i];
    }
}

void MSGFR::FindInterval( double &a, double &b )
{
    const double H0 = 0.1;           // Начальный шаг
    const double TAU = 2.0;          // Множитель увеличения шага
    const int MAX_ITER = 100;        // Максимальное число итераций

    double lambda = 0.0;             // Текущая точка
    double h = H0;                    // Текущий шаг

    double f_prev = f.funcInDirection( xk, lambda - h, sk );
    double f_curr = f.funcInDirection( xk, lambda, sk );
    double f_next = f.funcInDirection( xk, lambda + h, sk );
}

```

```

if ( f_prev < f_curr )
{
    h = -h;
}
else if ( f_next < f_curr ) { }
else
{
    a = lambda - h;
    b = lambda + h;
    return;
}

int iter = 0;
double lambda_prev = lambda;
double lambda_curr = lambda + h;
double f_curr_val, f_next_val;

while ( iter < MAX_ITER ) {
    f_curr_val = f.funcInDirection( xk, lambda_curr, sk );
    f_next_val = f.funcInDirection( xk, lambda_curr + h, sk );

    if ( f_curr_val < f_next_val ) {
        a = std::min( lambda_prev, lambda_curr + h );
        b = std::max( lambda_prev, lambda_curr + h );

        return;
    }
    lambda_prev = lambda_curr;
    lambda_curr = lambda_curr + h;
    h *= TAU;

    iter++;
}
a = 0.0;
b = 10.0;
}

double *MSGFR::Solver( )
{
    const double PI = 3.14159265359;
    std::ofstream file1( "out1.txt" );

```

```

file1 << "i\t" << "x\t" << "y\t" << "f\t" << "s1\t" << "s2\t" << "lambda\t" << "|xi - xi-1|\t" << "|yi - yi-1|\t" <<
"|fi - fi-1|\t" << "angle\t" << "grad1\tgrad2" << std::endl;
std::ofstream file2( "out2.txt" );
file2 << 0 << ' ' << x0[0] << ' ' << x0[1] << std::endl;
double a = -1, b = 1;
//FindInterval( a, b, x0 );
//lambdak = Parabola( eps1, a, b );
//xk = ops.AddVec( x0, ops.MultVecScal( sk, lambdak ) );
int k = 0;
while ( ops.DotProduct( sk, sk ) >= eps )
{
    if ( (k+1) % n1 == 0 )
    {
        sk = ops.MultVecScal( f.gradFunc( xk ), -1 );
    }
    FindInterval( a, b );
    lambdak = Parabola( eps1, a, b );
    //lambdak = GoldenRatio(eps1, a, b);
    for ( int i = 0; i < n; i++ ) xk_old[i] = xk[i];
    xk = ops.AddVec( xk, ops.MultVecScal( sk, lambdak ) );
    for ( int i = 0; i < n; i++ ) xk_1[i] = xk_old[i];
    for ( int i = 0; i < n; i++ ) sk_1[i] = sk[i];
    double wk = CalcW( );
    sk = ops.AddVec( ops.MultVecScal( f.gradFunc( xk ), -1 ), ops.MultVecScal( sk, wk ) );
    k++;
    std::cout << k << "\t";
    file1 << k << "\t";
    file2 << k << " ";
    for ( int i = 0; i < n; i++ )
    {
        std::cout << std::setprecision( 16 ) << xk[i] << ' ';
        file1 << std::setprecision( 16 ) << xk[i] << '\t';
        file2 << std::setprecision( 16 ) << xk[i] << ' ';
    }
    file1 << std::setprecision( 16 ) << f.func( xk ) << '\t' << sk[0] << '\t' << sk[1] << '\t' << lambdak << '\t';
    file1 << abs( xk[0] - xk_1[0] ) << '\t' << abs( xk[1] - xk_1[1] ) << '\t' << abs( f.func( xk ) - f.func( xk_1 ) ) <<
'\t';
    file1 << std::acos( ( xk[0] * sk[0] + xk[1] * sk[1] ) / ( sqrt( ops.DotProduct( xk, xk ) ) * sqrt( ops.DotProduct(
sk, sk ) ) ) * 180 / PI << '\t';
    file1 << f.gradFunc( xk )[0] << '\t' << f.gradFunc( xk )[1];
    std::cout << std::endl;
    file1 << std::endl;
    file2 << std::endl;
}

```

```

    }
    return xk;
}

double MSGFR::CalcW( )
{
    double a = ops.DotProduct( f.gradFunc( xk ), f.gradFunc( xk ) );
    double b = ops.DotProduct( f.gradFunc( xk_1 ), f.gradFunc( xk_1 ) );
    return a / b;
}

double MSGFR::Min( double eps, double an, double bn )
{
    double x, x1, x2;
    double delta = 0.5 * eps1;
    int n = 0;
    while ( bn - an >= eps )
    {
        n++;
        x = 0.5 * ( an + bn );
        x1 = x - 0.5 * delta;
        x2 = x + 0.5 * delta;
        double f1 = f.funcInDirection( xk, x1, sk );
        double f2 = f.funcInDirection( xk, x2, sk );
        if ( f1 < f2 )
        {
            bn = x2;
        }
        else if ( f1 > f2 )
        {
            an = x1;
        }
        else
        {
            break;
        }
    }
    return x;
}

double MSGFR::GoldenRatio( double eps, double an, double bn )
{

```

```

double x, x1, x2;
int n = 1;
double const C = 0.38196601125010515;
x1 = an + C * ( bn - an );
x2 = bn - C * ( bn - an );
double f1 = f.funcInDirection( xk, x1, sk );
double f2 = f.funcInDirection( xk, x2, sk );
while ( bn - an >= eps )
{
    x = 0.5 * ( an + bn );
    n++;
    if ( f1 < f2 )
    {
        bn = x2;
        x2 = x1;
        f2 = f1;
        x1 = an + C * ( bn - an );
        f1 = f.funcInDirection( xk, x1, sk );
    }
    else if ( f1 > f2 )
    {
        an = x1;
        x1 = x2;
        f1 = f2;
        x2 = bn - C * ( bn - an );
        f2 = f.funcInDirection( xk, x2, sk );
    }
    else
    {
        break;
    }
}
return x;
}

double MSGFR::Parabola( double eps, double an, double bn )
{
    double x1 = an, x2 = ( an + bn ) / 2.0, x3 = bn;
    double x = x2 + 10 * eps;
    double f0, f1 = f.funcInDirection( xk, x1, sk ), f2 = f.funcInDirection( xk, x2, sk ), f3 = f.funcInDirection( xk, x3,
sk );
    double numer, denom; // числитель, знаменатель
    int k = 0;

```



```

while ( fabs( x2 - x ) >= eps )
{
    numer = ( x2 - x1 ) * ( x2 - x1 ) * ( f2 - f3 ) - ( x2 - x3 ) * ( x2 - x3 ) * ( f2 - f1 );
    denom = ( x2 - x1 ) * ( f2 - f3 ) - ( x2 - x3 ) * ( f2 - f1 );
    if ( denom == 0 )
        x = x2;
    else
        x = x2 - 0.5 * numer / denom;
    f0 = f.funcInDirection( xk, x, sk );
    if ( x <= x2 )
        if ( f0 <= f2 )
        {
            x3 = x2;
            f3 = f2;
            x2 = x;
            f2 = f0;
        }
        else
        {
            x1 = x;
            f1 = f0;
        }
    else
        if ( f0 <= f2 )
        {
            x1 = x2;
            x2 = x;
            f1 = f2;
            f2 = f0;
        }
        else
        {
            x3 = x;
            f3 = f0;
        }
    k++;
}
std::cout << k << std::endl;
return x2;
}

```

5.8 Broyden.h

```
#pragma once
#include "Func.h"
#include "VecOperations.h"
#include "MatrixOperations.h"
class Broyden
{
    int n = 2;
    double *x0;
    double *xk;
    double *xk_1;
    double **etak;
    double lambdak;
    double eps = 1e-7;
    double eps1 = 1e-7;
    Func f;
    VecOperations opsVec;
    MatrixOperations opsMat;
public:
    void Init( int n1 );
    void FindInterval( double& a, double& b );
    double *Solver( );
    void CalcEtak( );
    double Min( double eps, double an, double bn );
    double GoldenRatio( double eps, double an, double bn );
    double Parabola( double eps, double an, double bn );
};
```

5.9 Broyden.cpp

```
#include "Broyden.h"
#include "iostream"
#include <iomanip>
#include <fstream>
#include <cmath>
void Broyden::Init( int n1 )
{
    n = n1;
    x0 = new double[2] { 0, -1 };
    xk = new double[n] { 0 };
    xk_1 = new double[n] { 0 };
    etak = new double *[n];
```

```

for ( int i = 0; i < n; i++ )
{
    etak[i] = new double[n];
    for ( int j = 0; j < n; j++ )
        etak[i][j] = ( i == j ) ? 1.0 : 0.0;
}

void Broyden::FindInterval( double &a, double &b )
{
    const double H0 = 0.1;          // Начальный шаг
    const double TAU = 2.0;         // Множитель увеличения шага
    const int MAX_ITER = 100;       // Максимальное число итераций

    double lambda = 0.0;           // Текущая точка
    double h = H0;                  // Текущий шаг

    double f_prev = f.funcInDirectionEta( xk, lambda - h, etak );

    double f_curr = f.funcInDirectionEta( xk, lambda, etak );
    double f_next = f.funcInDirectionEta( xk, lambda + h, etak );

    if ( f_prev < f_curr )
    {
        h = -h;
    }
    else if ( f_next < f_curr ) { }
    else
    {
        a = lambda - h;
        b = lambda + h;
        return;
    }

    int iter = 0;
    double lambda_prev = lambda;
    double lambda_curr = lambda + h;
    double f_curr_val, f_next_val;

    while ( iter < MAX_ITER ) {
        f_curr_val = f.funcInDirectionEta( xk, lambda_curr, etak );
        f_next_val = f.funcInDirectionEta( xk, lambda_curr + h, etak );
    }
}

```

```

        if ( f_curr_val < f_next_val ) {
            a = std::min( lambda_prev, lambda_curr + h );
            b = std::max( lambda_prev, lambda_curr + h );

            return;
        }
        lambda_prev = lambda_curr;
        lambda_curr = lambda_curr + h;
        h *= TAU;

        iter++;
    }
    a = 0.0;
    b = 10.0;
}

double *Broyden::Solver( )
{
    double a = 0, b = 0;
    xk = x0;
    int k = 0;
    const double PI = 3.14159265359;
    std::ofstream file1( "out1.txt" );
    file1 << "i\t" << "x\t" << "y\t" << "f\t" << "lambda\t" << "|xi - xi-1|\t" << "|yi - yi-1|\t" << "|fi - fi-1|\t" <<
"eta1\teta2\teta3\teta4\t" << "grad1\tgrad2" << std::endl;
    std::ofstream file2( "out2.txt" );
    file2 << 0 << ' ' << xk[0] << ' ' << xk[1] << std::endl;
    while ( opsVec.DotProduct( f.gradFunc( xk ), f.gradFunc( xk ) ) >= eps )
    {
        for ( int i = 0; i < n; i++ ) xk_1[i] = xk[i];
        FindInterval( a, b );
        lambdak = Parabola( eps1, a, b );
        //lambdak = GoldenRatio(eps1, a, b );
        xk = opsVec.AddVec( xk_1, opsVec.MultVecScal( opsMat.MultMatVec( etak, f.gradFunc( xk_1 ) ), -lambdak ) );
        CalcEtak( );
        k++;
        std::cout << k << "\t";
        file1 << k << "\t";
        file2 << k << " ";
        for ( int i = 0; i < n; i++ )
        {
            std::cout << std::setprecision( 16 ) << xk[i] << ' ';

```

```

        file1 << std::setprecision( 16 ) << xk[i] << '\t';
        file2 << std::setprecision( 16 ) << xk[i] << ' ';
    }
    std::cout << '\t';
    for ( int i = 0; i < n; i++ )
        for ( int j = 0; j < n; j++ )
            std::cout << std::setprecision( 16 ) << etak[i][j] << ' ';
    file1 << std::setprecision( 16 ) << abs( f.func( xk ) ) << '\t' << lambdak << '\t';
    file1 << abs( xk[0] - xk_1[0] ) << '\t' << abs( xk[1] - xk_1[1] ) << '\t' << abs( f.func( xk ) - f.func( xk_1 ) ) <<
'\t';
    file1 << etak[0][0] << '\t' << etak[0][1] << '\t' << etak[1][0] << '\t' << etak[1][1] << '\t' << f.gradFunc( xk )[0]
<< '\t' << f.gradFunc( xk )[1];
    std::cout << std::endl;
    file1 << std::endl;
    file2 << std::endl;

}
return xk;
}

void Broyden::CalcEtak( )
{
    double *deltaxk = opsVec.AddVec( xk, opsVec.MultVecScal( xk_1, -1 ) );
    double *deltagk = opsVec.AddVec( f.gradFunc( xk ), opsVec.MultVecScal( f.gradFunc( xk_1 ), -1 ) );
    double *buf = opsVec.AddVec( deltaxk, opsMat.MultMatVec( etak, opsVec.MultVecScal( deltagk, -1 ) ) );
    double denominator = 1 / opsVec.DotProduct( buf, deltagk );
    etak = opsMat.AddMatrix( opsMat.MultMatScal( opsMat.MultVecVecT( buf, buf ), denominator ), etak );
    delete[] deltaxk;
    delete[] deltagk;
    delete[] buf;
}

double Broyden::Min( double eps, double an, double bn )
{
    double x, x1, x2;
    double delta = 0.5 * eps;
    int n = 0;
    while ( bn - an >= eps )
    {
        n++;
        x = 0.5 * ( an + bn );
        x1 = x - 0.5 * delta;
        x2 = x + 0.5 * delta;
    }
}

```

```

        double f1 = f.funcInDirectionEta( xk, x1, etak );
        double f2 = f.funcInDirectionEta( xk, x2, etak );
        if ( f1 < f2 )
        {
            bn = x2;
        }
        else if ( f1 > f2 )
        {
            an = x1;
        }
        else
        {
            break;
        }
    }
    return x;
}

double Broyden::GoldenRatio( double eps, double an, double bn )
{
    double x, x1, x2;
    int n = 1;
    double const C = 0.38196601125010515;
    x1 = an + C * ( bn - an );
    x2 = bn - C * ( bn - an );
    double f1 = f.funcInDirectionEta( xk, x1, etak );
    double f2 = f.funcInDirectionEta( xk, x2, etak );
    while ( bn - an >= eps )
    {
        x = 0.5 * ( an + bn );
        n++;
        if ( f1 < f2 )
        {
            bn = x2;
            x2 = x1;
            f2 = f1;
            x1 = an + C * ( bn - an );
            f1 = f.funcInDirectionEta( xk, x1, etak );
        }
        else if ( f1 > f2 )
        {
            an = x1;
            x1 = x2;

```

```

        f1 = f2;
        x2 = bn - C * ( bn - an );
        f2 = f.funcInDirectionEta( xk, x2, etak );
    }
    else
    {
        break;
    }
}
return x;
}

double Broyden::Parabola( double eps, double an, double bn )
{
    double x1 = an, x2 = ( an + bn ) / 2.0, x3 = bn;
    double x = x2 + 10 * eps;
    double f0, f1 = f.funcInDirectionEta( xk, x1, etak ), f2 = f.funcInDirectionEta( xk, x2, etak ), f3 =
f.funcInDirectionEta( xk, x3, etak );
    double numer, denom; // числитель, знаменатель
    int k = 0;
    while ( fabs( x2 - x ) >= eps )
    {
        numer = ( x2 - x1 ) * ( x2 - x1 ) * ( f2 - f3 ) - ( x2 - x3 ) * ( x2 - x3 ) * ( f2 - f1 );
        denom = ( x2 - x1 ) * ( f2 - f3 ) - ( x2 - x3 ) * ( f2 - f1 );
        if ( denom == 0 )
            x = x2;
        else
            x = x2 - 0.5 * numer / denom;
        f0 = f.funcInDirectionEta( xk, x, etak );
        if ( x <= x2 )
            if ( f0 <= f2 )
            {
                x3 = x2;
                f3 = f2;
                x2 = x;
                f2 = f0;
            }
        else
        {
            x1 = x;
            f1 = f0;
        }
    }
    else

```

```

        if ( f0 <= f2 )
        {
            x1 = x2;
            x2 = x;
            f1 = f2;
            f2 = f0;
        }
        else
        {
            x3 = x;
            f3 = f0;
        }
        k++;
    }
    std::cout << k << std::endl;
    return x2;
}

```

5.10 VecOperations.h

```

#pragma once
#include <iostream>
class VecOperations
{
public:
    int n= 2;
    double *MultVecScal( double *vec, double scal );
    double *AddVec( double *vec1, double *vec2 );
    double DotProduct( double *vec1, double * vec2 );
};

```

5.11 VecOperations.cpp

```

#include "VecOperations.h"
#include <iostream>
double *VecOperations::MultVecScal( double *vec, double scal )
{
    double *res = new double[n];
    for ( int i = 0; i < n; i++ )
        res[i] = vec[i] * scal;
}

```



```
    return res;
}

double *VecOperations::AddVec( double *vec1, double *vec2 )
{
    double *res = new double[n];
    for ( int i = 0; i < n; i++ )
        res[i] = vec1[i] + vec2[i];
    return res;
}

double VecOperations::DotProduct( double *vec1, double *vec2 )
{
    double res = 0;
    for ( int i = 0; i < n; i++ )
        res += vec1[i] * vec2[i];
    return res;
}
```